

Computer Vision - Task 1

Top-View

The goal of the top-view module was not only to detect the table, but also to orient it correctly so that the real long side of the table appears horizontally in the final rectified image. The table area is first isolated by thresholding the blue cloth in HSV space, extracting the largest contour, approximating it with **approxPolyDP**, and then rectifying it with a perspective transform using **getPerspectiveTransform** and **warpPerspective**.

At first, we tried to infer the long side from the image dimensions, but this was unreliable because in frontal views the table can appear vertically even when it is physically longer horizontally. We also explored detecting the middle pockets, since only the long rails contain them, but that approach was not robust enough in practice because those regions were visually inconsistent across images.

We therefore used the white rail markers as the main geometric cue. The long sides of the table contain two triplets of white dots, while the short sides contain only one, so detecting these triplets gives a more stable way to identify the real long edges. The dots are extracted with a white HSV mask, merged when detections are too close, and filtered by distance to the table border so that only markers near the rails are kept.

After that, each detected point is assigned to its nearest table edge, and all valid 3-point combinations are tested on that side. A triplet is accepted only if the points are consecutive, roughly equally spaced, and approximately collinear; line consistency is evaluated with **cv2.fitLine**, which fits a line to a 2D set of points.

The side with the strongest long-edge evidence is then chosen as the main edge. From that edge, the four table corners are ordered consistently, which allows the perspective transform to generate a top view with the correct orientation.

Finally, a fixed padding is added during the warp so that the outer table border is preserved, not only the blue playing area. A last cropping step uses a grey mask over the warped image to recover the visible border more accurately and remove excess padding, producing the final top-view result.

Bounding Boxes

The algorithm first isolates the pool table, since detecting balls over the whole image would introduce much more background noise.

Next, it estimates the physical scale of the scene from the detected table size. That scale is then used to infer the expected ball radius in pixels, so the later detection stage is guided by geometry rather than fixed values.

The central idea is simple: detect the table, define a safe working area inside it, enhance that region, and search only there for circular objects consistent with pool

balls. This makes the detection more stable by avoiding walls, cues, pockets, border reflections, and other non-ball regions.

After masking the table, the image is filtered and contrast-enhanced so that circular objects stand out more clearly. This is a constrained detection approach: instead of asking where balls might be anywhere in the image, the algorithm searches only for ball-sized circles inside the playable table region.

The estimated table width provides a scene-specific scale, allowing Hough circle detection to use more appropriate radius limits and spacing constraints. This is especially important when the apparent ball size changes across images or when background clutter is present.

If circles are found inside the safe zone, the algorithm draws bounding boxes and center points, counts the detected balls, and returns both the cropped image of each ball and its corresponding normalized bounding-box coordinates. In this way, the method validates the table structure before detecting the balls, making it more reliable for controlled pool-table scenes.

Number Detection

The number-identification stage starts by assigning the cue ball and the 8-ball first, since they are visually the most distinctive cases. The cue ball is selected as the crop with the strongest dominance of white pixels and the lowest amount of non-cloth color, which helps handle dirty or yellowish whites caused by lighting and material variation. The 8-ball is then identified as the remaining crop with the highest proportion of valid black pixels.

After that, the remaining balls are separated into solids and stripes. This is done mainly through the percentage of white pixels inside each crop: balls with a significant white area are first treated as stripe candidates, and ambiguous cases are refined using a color-to-white ratio so that solids with exposed highlights or reflections are not wrongly classified as stripes.

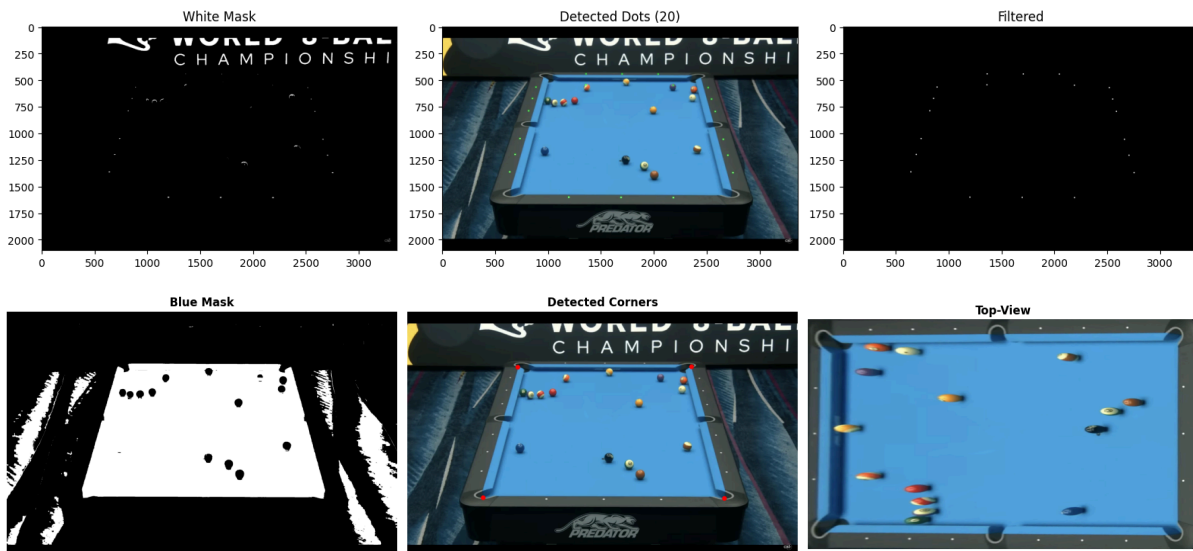
Once the ball type is known, the algorithm compares each crop against a calibrated color palette. Each ball crop is blurred with a median filter and analyzed in HSV space, which is useful here because HSV separates hue from saturation and brightness, while median filtering reduces noise without strongly damaging object boundaries. The color descriptor is built from the central ball region, ignoring most white stripe areas and dark shadows, and uses robust statistics such as the median hue and high-percentile saturation and value.

The final assignment is performed globally rather than independently. Each crop is scored against all valid color-number candidates, the matches are sorted from best to worst, and the algorithm greedily locks the lowest-distance valid assignments while preventing duplicate ball numbers. In practice, this means that if two crops compete for the same color, the one with the better score keeps that label and the other is reassigned to the next best available option.

Results

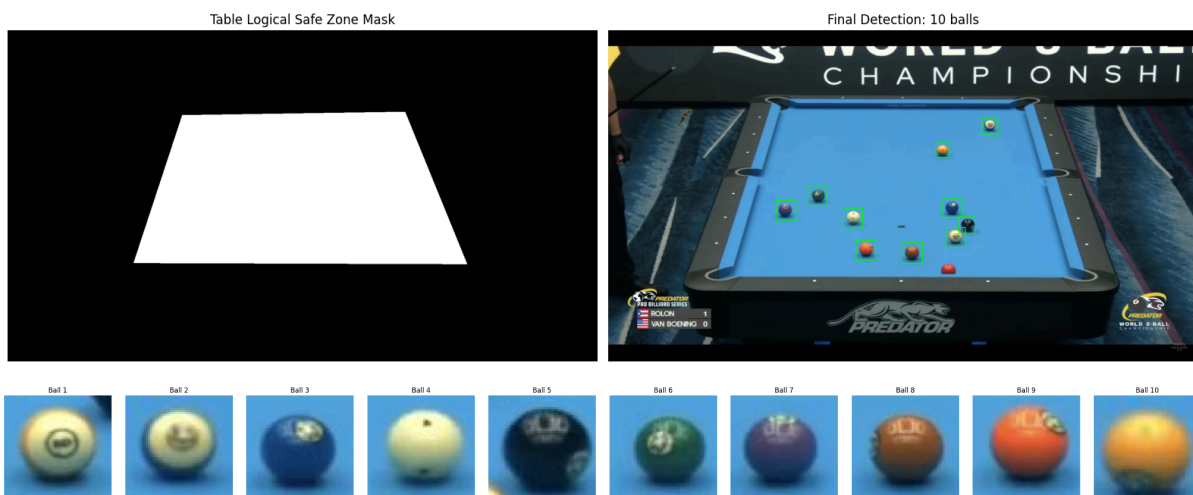
Top-View

For image path **'3f_png.rf.81c7e132365ef95bb19380ca389025f6.jpg'**, the obtained results are shown below. In addition to the final top-view image, the intermediate outputs are included to illustrate the processing stages, particularly the detected white markers used to identify the long table sides.



Bounding Boxes

For the bounding-box detection of image **'155_png.rf.f3ae78698646b2d6f43fe88d2d1752f4'** the safe-zone mask used to define the valid table region is shown, together with the final detection result and the bounding box of each detected ball.



Number Detection

Number-detection debugging was based mainly on printed white percentages and color scores, which were used to refine the palette and thresholds. Since each detected crop was also compared with its reference ball, no extra visual results are shown here; the final outputs are provided in the JSON file.

Team Members

Gonçalo Marques - up202206205

Miguel Guerrinha - up202205038

Miguel Duarte - up202206102

Rui Cruz - up202208011